MODULE 05 · DOMAIN 3 - CORE TERRAFORM WORKFLOW

The Core Workflow: init, plan, apply, destroy

The four commands, the flags that change what they do, and the two that HashiCorp warns you about.

DOMAIN 3 of 8 COURSE WEIGHT 16% MODULE 5 of 27

admin@sthithapragna.com

Write, plan, apply. Everything else is a **variation** on those three steps.

admin@sthithapragna.com

Write, plan, apply

01

Write

Author configuration describing the infrastructure you want.

02

Init

Prepare the working directory. Backend, providers, modules.

03

Plan

Preview the changes. Nothing is altered.

04

Apply

Execute the plan against the provider APIs.

ON THE EXAM

Init runs once per working directory, and again whenever its inputs change. Plan and apply are the loop you live in.

What terraform init actually does

- **Initialises the backend**, so Terraform knows where state lives before anything else happens.
- **Downloads and installs providers** named in required_providers, writing the dependency lock file.
- **Downloads modules** referenced by module blocks, into the .terraform directory.
- **Safe to run repeatedly.** It will not destroy anything and will not change existing state.
- It does **not** create infrastructure, read your variables, or contact your cloud provider's resource APIs.

> ON THE EXAM
>
> Three responsibilities: backend, providers, modules. Almost every init question is about one of those three changing.

When you must run init again

- **You added a provider,** or changed a version constraint so the locked version no longer fits.
- **You added or changed a module block,** including its source or version.
- **You changed the backend configuration,** or added one where there was none.
- You cloned the repository, or deleted the **.terraform** directory. It is disposable, so recreate it.
- Other commands **detect this and tell you**. The error asking you to run init is not a bug.

> ON THE EXAM
>
> Backend, providers, modules. If one of those three changed, init needs running. Nothing else requires it.

The init flags worth knowing

FLAG	WHAT IT DOES
`-upgrade`	Re-resolve providers and modules against current constraints, ignoring the lock
`-reconfigure`	Reinitialise the backend, discarding existing configuration. No state migration
`-migrate-state`	Reinitialise the backend and offer to copy existing state across
`-backend-config=...`	Supply backend settings from a file or key-value pair at init time
`-input=false`	Never prompt. Essential in automation, where there is nobody to answer
`-get=false`	Skip module installation. Rarely useful outside constrained environments

ON THE EXAM

-reconfigure and -migrate-state both change backends. Only one offers to bring your state with it.

What terraform plan does, precisely

- **Reads the current state of existing remote objects,** so state is up to date. This is the refresh.
- **Compares the configuration against the prior state,** noting every difference.
- **Proposes change actions** that would make reality match the configuration.
- **Plan alone carries out nothing.** It is a preview, and running it is always safe.
- If nothing differs, Terraform reports that no actions need to be taken.

> ON THE EXAM
>
> Three operations in order: refresh, compare, propose. That is module four's three-input model as a command.

Speculative plan or saved plan?

SPECULATIVE

terraform plan, with no -out

- A description of the effect
- No intent to apply it
- Ideal before code review
- Cannot be handed to apply

SAVED

terraform plan -out=tfplan

- An opaque file on disk
- Passed to apply as an argument
- Applied exactly as written, no prompt
- Intended mainly for automation

THE TIEBREAKER

A saved plan applies without asking, because you already approved it when you generated it. That is the point and the risk.

A saved plan file is a sensitive artefact

- **The plan file contains your full configuration**, every planned value, and all the input variables.
- **Sensitive data is stored in cleartext,** even where Terraform obscured it in terminal output.
- HashiCorp's guidance: treat saved plan files as **potentially sensitive artefacts**.
- So do not publish them as build artefacts, and do not retain them longer than the apply needs.
- **Never name it with a .tf suffix.** Terraform would try to parse it as configuration and every later command breaks.

> ON THE EXAM
>
> The same lesson as state in module four. Obscured on screen has never meant absent from the file.

The three planning modes

MODE	ITS GOAL
`Normal`	Change the remote system to match your configuration
`-destroy`	Destroy every remote object, leaving an empty state
`-refresh-only`	Update state and outputs to match reality. Change no infrastructure

ON THE EXAM

Mutually exclusive. Choosing one disables normal mode, and you cannot combine two.

Options that change the plan

OPTION	EFFECT
`-refresh=false`	Skip the refresh. Faster, and external changes become invisible
`-replace=ADDRESS`	Plan to destroy and recreate one instance. Available from v0.15.2
`-target=ADDRESS`	Restrict planning to an address and everything it depends on
`-var 'name=value'`	Set one input variable. No spaces around the equals sign
`-var-file=FILE`	Set many from a .tfvars file. Preferred over -var for anything complex

ON THE EXAM

These work on both plan and apply. -replace cannot be combined with -destroy, and -refresh=false is invalid in refresh-only mode.

-target: read the warning

- It selects the address you name **plus everything it depends on**, directly or indirectly.
- HashiCorp: for **exceptional circumstances only**, such as recovering from mistakes or working around limitations.
- **Not recommended for routine operations.** It causes undetected drift and confusion about what state means.
- The documented alternative: **break a large configuration into several smaller ones** applied independently.
- Then share values between them with **data sources**, so the architecture stays connected.

> ON THE EXAM
>
> If a scenario uses -target as a routine habit, that habit is the wrong answer. It is a recovery tool.

-replace, and the command it superseded

- **-replace=ADDRESS** plans to destroy and recreate one instance, even though nothing in configuration changed.
- The use case: a remote object has become **degraded** and you want a fresh one built identically.
- It appears **in the plan**, so you see the replacement before approving it. Repeat the flag for several objects.
- **terraform taint** did this by editing state directly, with no preview. It is deprecated.
- -replace arrived in **v0.15.2**. Use taint only on versions older than that.

> ON THE EXAM
>
> The improvement is visibility. taint changed state and hoped; -replace shows you the replacement in a plan first.

terraform apply, in two forms

- **With no plan file:** apply generates a fresh plan, shows it, and prompts you to approve.
- **With a plan file:** apply executes that plan exactly as written, and does not prompt.
- **-auto-approve** skips the prompt in the first form. Standard in automation, dangerous at a keyboard.
- Apply accepts the planning modes and options too, so **apply -destroy** and **apply -refresh-only** are both valid.
- A saved plan can go **stale**. Other changes may have landed since it was generated.

> ON THE EXAM
>
> Apply never prompts for a saved plan. The approval is considered to have happened when the plan was created.

terraform destroy

- **Identical to apply -destroy.** It is a convenience alias for destroy planning mode.
- It destroys in **reverse dependency order**, using the dependency metadata state retains.
- Legitimate use: **transient environments** that stop being useful once the work is done.
- **prevent_destroy** on a lifecycle block makes Terraform refuse. Module eleven covers it.
- It shows a plan and prompts, exactly like apply. **-auto-approve** skips that, and rarely should.

> ON THE EXAM
>
> Destroy is not a separate mechanism. Knowing it is apply in a different mode explains why its flags are the same.

Flags for running in automation

FLAG	WHY A PIPELINE NEEDS IT
`-input=false`	There is no human to answer a prompt. Missing values become errors
`-auto-approve`	Skips confirmation when applying without a saved plan
`-detailed-exitcode`	0 no changes, 1 error, 2 changes present. Lets a pipeline branch on the result
`-lock-timeout=DURATION`	Wait for a held lock instead of failing immediately
`-no-color`	Strips terminal formatting for systems that cannot render it
`-parallelism=n`	Limits concurrent operations while walking the graph. Defaults to 10

ON THE EXAM

-detailed-exitcode is the one people forget. Without it, a plan with changes and a plan with none both exit zero.

Reading the scenario

THE WORDING SAYS	IT IS POINTING AT
A new module block was added and plan fails	Run terraform init again
Speed up applies by working on one part at a time	-target misused. Split the configuration instead
A degraded instance must be rebuilt identically	-replace, not taint and not destroy
The pipeline must know whether changes exist	-detailed-exitcode
Approve now, apply after review	A saved plan with -out, and re-check before applying
Changed outside Terraform, record it only	Refresh-only mode, not normal mode

ON THE EXAM

Ask which of the three steps is under stress: preparing the directory, previewing the change, or executing it.

A configuration has grown to roughly two thousand resources covering networking, databases and twelve application services. Applies take a long time, so the team has adopted a habit of running apply with -target set to whichever service they are working on.

Over six months, three incidents have been traced to resources that were never updated because nobody targeted them. The team asks how to make targeting safer.

What should they be advised?

A Target the parent module rather than individual resources, so dependencies are included

B Stop routine targeting and split the configuration into smaller ones, sharing values via data sources

C Continue targeting, but run a full untargeted apply weekly to catch stragglers

D Raise -parallelism so full applies complete fast enough to remove the incentive

◆ PAUSE AND DECIDE

admin@sthithapragna.com

What should they be advised?

A Target the module instead

Targeting a module does widen the selection, and it is still targeting. The undetected drift the documentation warns about is unchanged.

B Split the configuration

Correct, and it is the alternative HashiCorp names explicitly: smaller independently applied configurations, connected by data sources.

C Weekly full apply

Reduces the window without closing it. It also means a weekly apply carrying six days of unreviewed change, which is a worse blast radius.

D Raise parallelism

Treats a design problem as a throughput problem. Provider rate limits usually bite first, and the configuration is still one unit of change.

THE SIGNAL

The question asks how to make targeting safer. The documented answer is to stop needing it.

A pipeline runs terraform plan -out=tfplan, then publishes tfplan as a build artefact so reviewers can download it. The artefact store is readable by everyone in the engineering organisation.

The configuration provisions a database whose administrator password is supplied through an input variable marked sensitive. Plan output in the build log correctly shows the value as sensitive.

What is the problem with this pipeline?

A There is no problem. The sensitive marking protects the value in the plan file

B The plan file stores the value in cleartext and is readable by the whole organisation

C Plan files cannot be published as artefacts because apply requires them in place

D The problem is the build log, since plan output always contains variable values

◆ PAUSE AND DECIDE

What is the problem with this pipeline?

A. No problem

Repeats the misconception from module four in a new location. sensitive governs display, and the plan file is storage, not display.

B. Cleartext, and readable

Correct. HashiCorp states that sensitive data is saved in cleartext in the plan file, even when obscured in terminal output, and calls plan files potentially sensitive artefacts.

C. Cannot be published

False. Moving plan files between steps is normal and is what the two-step workflow is for. Where it goes and who can read it is the issue.

D. The build log

The scenario says the log correctly showed the value as sensitive. The one component behaving properly is being blamed.

THE SIGNAL

The same pattern as state: obscured on screen, cleartext in the file. Two files now, one rule.

A change control process requires that plans are generated on Monday, reviewed by a board on Wednesday, and applied on Friday using the saved plan file so that what was approved is exactly what runs.

On Thursday, an unrelated team applies a change to the same configuration to resolve an incident. The Friday apply is run against Monday's saved plan file.

What should the team expect?

A Apply will prompt for confirmation because the plan is no longer current

B Apply may fail or produce an unintended result, because the saved plan reflects Monday's state

C Apply will silently regenerate the plan, since saved plans expire after 24 hours

D Apply will merge Monday's plan with Thursday's changes automatically

◆ PAUSE AND DECIDE

admin@sthithapragna.com

What should the team expect?

A **It will prompt**

Apply never prompts for a saved plan. The approval is treated as having happened when the plan was generated, which is the entire premise here.

B **May fail or misfire**

Correct. HashiCorp warns that changes made in the meantime can make the final effect differ from what an earlier plan indicated, and that you should re-check the final plan.

C **Plans expire**

Invents a lifetime. A plan file has no expiry. It is stale the moment anything changes, and it never announces that.

D **It merges**

Terraform has no merge concept for plans. A plan is a fixed set of actions computed against one moment.

THE SIGNAL

The process is well intentioned and its guarantee is the wrong one. It guarantees what runs, not that what runs is still correct.

A virtual machine managed by Terraform has become unresponsive after a failed in-guest upgrade. Its configuration is correct and unchanged, and a freshly built instance from the same configuration would resolve the problem.

Terraform plans no changes, because from the provider's point of view the instance still matches the configuration.

What should the engineer run?

A terraform apply -replace=ADDRESS for that instance

B terraform destroy followed by terraform apply

C terraform apply -refresh-only, so Terraform notices the instance is degraded

D Edit the configuration to force a difference, apply, then revert it

◆ PAUSE AND DECIDE

What should the engineer run?

A apply -replace

Correct. This is exactly the documented use case: a degraded remote object replaced by an identical one, and the replacement appears in the plan for approval.

B destroy then apply

Destroys everything in the configuration, not one instance. A vastly larger blast radius for a single sick machine.

C apply -refresh-only

Refresh-only records reality into state and changes no infrastructure. The provider reports the instance as healthy, so it would find nothing.

D Fake a change

Works, and leaves two misleading commits in history and a window where configuration does not describe intent. -replace exists to avoid exactly this.

THE SIGNAL

Terraform cannot see in-guest failure. -replace is how you tell it something is wrong that the provider API cannot report.

> An engineer adds a module block sourcing a networking module from the private registry, then runs terraform plan. Terraform stops with an error saying the module is not installed and that she should run terraform init.
>
> She is confident nothing else changed, and asks whether running init will affect the twelve resources this configuration already manages.

What is the correct answer?

A. Init may recreate resources, so a maintenance window should be booked first

B. Init only prepares the working directory. It creates and destroys nothing

C. Init will refresh state against the provider, which may show unexpected drift

D. Init should be avoided here; terraform get installs modules without other side effects

◆ PAUSE AND DECIDE

What is the correct answer?

A Book a window

Init touches no infrastructure at all. Caution is reasonable in general and is misdirected at the one command that cannot break anything.

B Prepares only

Correct. Init handles backend, providers and modules. It is safe to run repeatedly, and the error message is a prompt rather than a warning.

C Init refreshes state

Refresh belongs to plan and apply. Init initialises the backend so state can be reached; it does not read remote objects.

D Use terraform get

terraform get does install modules, and it does not handle providers or the backend. It solves a narrower problem for no benefit here.

THE SIGNAL

Init's three responsibilities are backend, providers and modules. None of them touch a running resource.

Module 5 recap

- **Init handles backend, providers and modules.** It creates nothing and is safe to repeat.
- **Plan refreshes, compares, proposes.** Alone it changes nothing at all.
- **A saved plan applies without a prompt,** and can go stale. Re-check before applying.
- **Plan files hold secrets in cleartext,** exactly like state. Never publish them freely.
- **-target is a recovery tool.** Routine use is the wrong answer; split the configuration instead.
- **-replace rebuilds a degraded object** and shows it in the plan. taint is deprecated.

> ON THE EXAM
>
> Three planning modes: normal, destroy, refresh-only. Mutually exclusive, and destroy is just apply in a different mode.